

Sri Lanka Institute of Information Technology

Fundamentals of Data Mining (IT3051)



Technical Report

Bank Marketing Term Deposit Predictor

An end-to-end data mining system for predicting term deposit subscription from telemarketing campaign data

Group Name - **Data Minds**

Student ID	Student Name
IT23859456	Gunathilaka W.A.D.S.
IT23857780	Edirisinghe E.M.K.L.
IT23557024	Illesinghe E.S.
IT23544918	Kaluarachchi R.G.

Submission date: 06 October 2026

Source code: https://github.com/Ranudi-Kaluarachchi/FDM-Bank_Marketing.git

Declaration

We declare that this report and the accompanying software are our own work, that the work was divided equally among the four group members as described in Section 12, and that all external sources, datasets and libraries are acknowledged in the References.

Abstract

Direct-marketing campaigns are expensive because most calls do not lead to a sale. Using the UCI Bank Marketing dataset (Moro, Cortez & Rita, 2014) of 45,211 telemarketing contacts made by a Portuguese bank, we built a complete data mining system that predicts whether a client will subscribe to a term deposit. Only 11.7% of contacts subscribed, so the problem is a highly imbalanced binary classification task. Following the CRISP-DM process, the raw data was cleaned and validated, the leaking *duration* attribute was removed, five features were engineered, and six classifiers (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, K-Nearest Neighbours and Gaussian Naive Bayes) were tuned with stratified cross-validation inside a leakage-free scikit-learn pipeline.

Gradient Boosting was selected by cross-validated ROC-AUC (0.795) and achieved a held-out test ROC-AUC of **0.806** and PR-AUC of 0.466 (random baseline 0.117). With a decision threshold of 0.62 tuned on out-of-fold training predictions it reaches $F1 = 0.487$ (precision 45%, recall 53%). Ranking clients by the model and calling the top 10% reaches 44.4% of all subscribers, a 4.4x lift over random calling. The model is deployed behind a FastAPI REST service and a React dashboard that supports single-client prediction, CSV batch scoring and interactive model and data insights.

Keywords: data mining, classification, class imbalance, gradient boosting, CRISP-DM, bank marketing, FastAPI, React

Table of Contents

Declaration	2
Abstract	2
Table of Contents	3
1. Introduction.....	5
1.1 Background.....	5
1.2 Problem statement.....	5
1.3 Objectives	5
1.4 Scope	5
2. Dataset Description	5
3. Methodology	6
4. Data Preprocessing.....	7
4.1 Data cleaning	7
4.2 Avoiding target leakage.....	8
4.3 Feature engineering	8
4.4 Outliers, scaling and encoding.....	8
5. Exploratory Data Analysis.....	8
6. Data Mining Techniques and Modelling.....	11
6.1 Experimental design	11
6.2 Algorithms	11
7. Evaluation and Results	12
7.1 Evaluation metrics	12
7.2 Model comparison.....	12
7.3 ROC and precision-recall curves	14
7.4 Confusion matrices.....	15
7.5 Decision threshold tuning	15
7.6 Feature importance.....	16
7.7 Business evaluation: cumulative gains	17
8. System Design and Implementation	18
8.1 Architecture.....	18
8.2 Technology stack	18
8.3 Repository structure.....	19
8.4 Backend REST API	19
8.5 Frontend dashboard	20
9. Testing and Validation	24

10. Deployment and User Guide	25
11. Discussion	25
11.1 Business insights and recommendations	25
11.2 Limitations	26
11.3 Future work	26
12. Individual Contributions	26
12.1 Data Acquisition, Cleaning and Validation	27
12.2 Exploratory Data Analysis and Feature Engineering	27
12.3 Model Training, Tuning and Evaluation.....	28
12.4 System Implementation (Backend & Frontend) and Testing	29
12.5 Individual Task Allocation (For Reference).....	29
13. Conclusion	30
References.....	30
Appendix A – Per-model Confusion Matrices	31
Appendix B – Configuration and Reproducibility	34

1. Introduction

1.1 Background

Banks promote products such as term deposits through outbound telemarketing. Each call costs agent time and, when repeated, can annoy customers. If the bank could predict which clients are likely to subscribe, it could prioritise those clients, reduce the number of unproductive calls and increase the campaign's conversion rate. This is a classic predictive data mining problem: learn from the outcomes of past campaigns to score future contacts.

1.2 Problem statement

Given a client's demographic, financial and campaign-contact attributes that are known **before** a call is made, predict whether the client will subscribe to a term deposit (target y = yes/no), and expose this prediction in a usable application for marketing staff.

1.3 Objectives

1. Acquire the public UCI Bank Marketing dataset and assess its quality.
2. Clean, validate and prepare the data, avoiding target leakage.
3. Explore the data to identify the factors associated with subscription.
4. Engineer informative features and handle outliers and class imbalance.
5. Train, tune and compare at least five classification algorithms using appropriate metrics for imbalanced data.
6. Select the best model, tune its decision threshold and explain which features drive its predictions.
7. Deploy the model through a REST API and an interactive web dashboard, with automated tests.
8. Translate the results into actionable recommendations for the marketing team.

1.4 Scope

The system covers the full pipeline from raw data download to a working web application. It uses the `bank-full.csv` file (May 2008 – November 2010). Real-time integration with a bank CRM, user authentication and cloud hosting are outside the scope of this project and are listed as future work.

2. Dataset Description

The dataset was published by Moro, Cortez and Rita (2014) and is available from the UCI Machine Learning Repository under a CC BY 4.0 licence. Each row is one client contacted during a campaign. The file contains **45,211 rows and 17 columns** (16 input attributes and the target). It is downloaded automatically by `ml/download_data.py`.

Table 3: Attributes of bank-full.csv

Attribute	Type	Description	Use in model
age	Numeric	Client age in years	Used
job	Categorical (12)	Type of job (admin., blue-collar, student, ...)	Used
marital	Categorical (3)	Marital status (divorced includes widowed)	Used
education	Categorical (4)	primary, secondary, tertiary, unknown	Used
default	Binary	Has credit in default?	Used
balance	Numeric	Average yearly balance (EUR)	Used
housing	Binary	Has a housing loan?	Used

loan	Binary	Has a personal loan?	Used
contact	Categorical (3)	Contact communication type (cellular, telephone, unknown)	Used
day	Numeric	Last contact day of the month	Used
month	Categorical (12)	Last contact month of the year	Used
duration	Numeric	Last contact duration in seconds	Removed (leakage)
campaign	Numeric	Number of contacts during this campaign (incl. last)	Used
pdays	Numeric	Days since last contact in a previous campaign (-1 = never)	Replaced by engineered features
previous	Numeric	Number of contacts before this campaign	Used
poutcome	Categorical (4)	Outcome of the previous campaign	Used
y	Binary	Did the client subscribe to a term deposit?	Target

Table 4: Summary statistics of numeric attributes

Attribute	Min	Median	Mean	Max	Std
age	18	39	40.94	95	10.62
balance	-8,019	448	1,362.27	102,127	3,044.77
day	1	16	15.81	31	8.32
campaign	1	2	2.76	63	3.10
pdays	-1	-1	40.20	871	100.13
previous	0	0	0.58	275	2.30

The numeric attributes are strongly right-skewed: **balance** has a median of 448 EUR but a maximum of 102,127 EUR and negative values down to -8,019 EUR; **campaign** reaches 63 calls and **previous** 275 contacts. The target is highly imbalanced, with 5,289 subscribers (11.7%) against 39,922 non-subscribers.

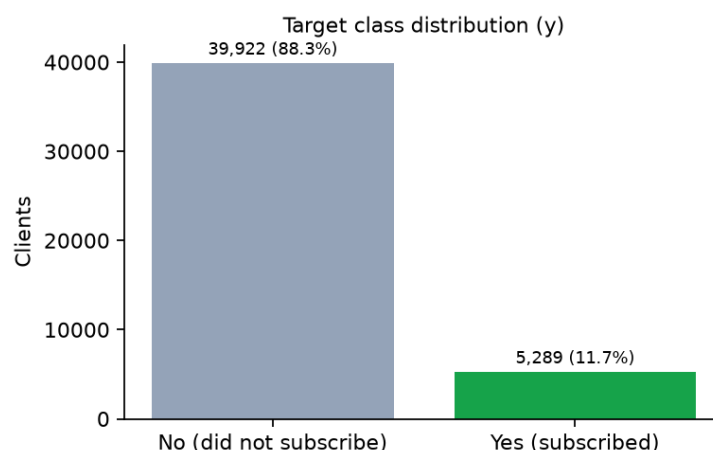


Figure 1: Distribution of the target variable

3. Methodology

The project followed the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology (Chapman et al., 2000). The table below maps each phase to the work done and the code that implements it.

Table 5: CRISP-DM phases and their implementation

CRISP-DM phase	Activities in this project	Implementation
Business understanding	Define the prediction goal, success metrics (ROC-AUC, F1, lift) and constraints (only pre-call data).	Section 1
Data understanding	Download data, profile attributes, check quality, exploratory analysis.	<code>download_data.py</code> , <code>eda.py</code>
Data preparation	Cleaning, validation, leakage removal, feature engineering, outlier capping, scaling, encoding.	<code>preprocess.py</code> , <code>transformers.py</code>
Modelling	Six algorithms, grid search with 3-fold stratified CV, class weighting.	<code>train.py</code>
Evaluation	Hold-out test metrics, ROC/PR curves, confusion matrices, threshold tuning, permutation importance, cumulative gains.	<code>train.py</code> , <code>report.py</code>
Deployment	Serialised pipeline served by a REST API and React dashboard; automated tests.	<code>backend/</code> , <code>frontend/</code>

The whole data mining process is reproducible with one command (`python -m ml.run_pipeline`), which runs download → cleaning → EDA → training and writes all artifacts. A fixed random seed (42) makes the data split and models deterministic.

4. Data Preprocessing

4.1 Data cleaning

Data cleaning is implemented in `ml/preprocess.py`. Every step is logged to `artifacts/cleaning_report.json`, which is also displayed in the dashboard.

Table 6: Data cleaning steps

#	Step	What was done	Result
1	Normalise text	Trim whitespace and lower-case column names and categorical values so that “Yes”, “ yes” and “YES” are identical.	Consistent categories
2	Missing values	Count NaNs; numeric NaNs are median-imputed and categorical NaNs set to “unknown”. Existing “unknown” placeholders are kept as their own category.	0 NaNs. “unknown”: job 288, education 1,857, contact 13,020, poutcome 36,959
3	Remove duplicates	Drop exact duplicate rows.	0 rows removed
4	Validate domain rules	age 18–100, day 1–31, valid month name, campaign ≥ 1, pdays = -1 or ≥ 0, previous ≥ 0, binary fields yes/no.	0 rows removed
5	Remove leakage	Drop <code>duration</code> : the call length is only known after the call ends and is almost a proxy for the outcome.	16 → 15 inputs
6	Encode target	y: yes → 1, no → 0.	Positive rate 11.7%

The dataset turned out to be of high technical quality: no missing values, duplicates or rule violations were found, so all 45,211 rows were retained. The main quality issue is semantic: “unknown” placeholders, especially in `poutcome` (36,959 rows, mostly clients never contacted before) and `contact` (13,020 rows). Because these groups have very different subscription rates (e.g. `contact` = unknown subscribes at only 4.1%), they carry signal and were kept as an explicit category rather than imputed or deleted.

4.2 Avoiding target leakage

Two forms of leakage were prevented. First, the `duration` attribute was removed because the dataset authors note it strongly determines the outcome but is unknown before the call, so a model using it would look excellent offline but be useless in practice. Second, every learned transformation (outlier limits, scaling statistics, one-hot categories) is fitted **inside** the scikit-learn pipeline, so during cross-validation it only sees the training folds, and the test set is never used for any fitting or tuning.

4.3 Feature engineering

`ml/transformers.py` defines a stateless `FeatureEngineer` transformer that derives five features from the raw inputs:

Table 7: Engineered features

Feature	Definition	Rationale
previously_contacted	1 if pdays $\neq$ -1, else 0	pdays = -1 is a code, not a number. Previously contacted clients subscribe at 23.1% vs 9.2%.
days_since_prev	pdays if contacted, else 0	Keeps recency information without the misleading -1.
log_balance	$\text{sign}(b) \cdot \log(1 + b)$	Compresses the long tail of balance while keeping negative balances negative.
total_contacts	campaign + previous	Total contact intensity across campaigns.
age_group	18-25, 26-35, 36-45, 46-55, 56-65, 65+	Captures the non-linear U-shaped effect of age.

The original `pdays` column is dropped after these features are created, giving 9 numeric and 10 categorical model inputs.

4.4 Outliers, scaling and encoding

- **Outliers:** a custom `QuantileCapper` winsorises each numeric feature to its 1st and 99th percentile learned on the training data. Extreme values (e.g. a balance of 102,127 EUR or 63 calls) are clipped rather than deleted, so no information about the client is lost.
- **Scaling:** numeric features are standardised (zero mean, unit variance) with `StandardScaler`, which distance-based (KNN) and linear (Logistic Regression) models require.
- **Encoding:** categorical features are one-hot encoded with `handle_unknown='ignore'`, so an unseen category at prediction time does not crash the service.

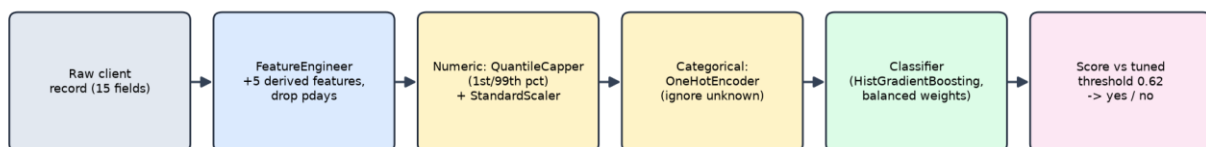


Figure 2: The deployed scikit-learn pipeline: raw record in, decision out

Because the saved artifact is the **entire pipeline**, the API receives the same 15 raw fields a user types into the form, and training and inference apply identical transformations.

5. Exploratory Data Analysis

`ml/eda.py` computes the subscription rate for every categorical attribute, an age histogram, the effect of the number of calls and the correlation of numeric attributes with the target. The results are saved to

`artifacts/insights.json` and rendered interactively on the dashboard's EDA page, which also highlights the strongest findings automatically. In the charts below the dashed line is the overall subscription rate (11.7%).

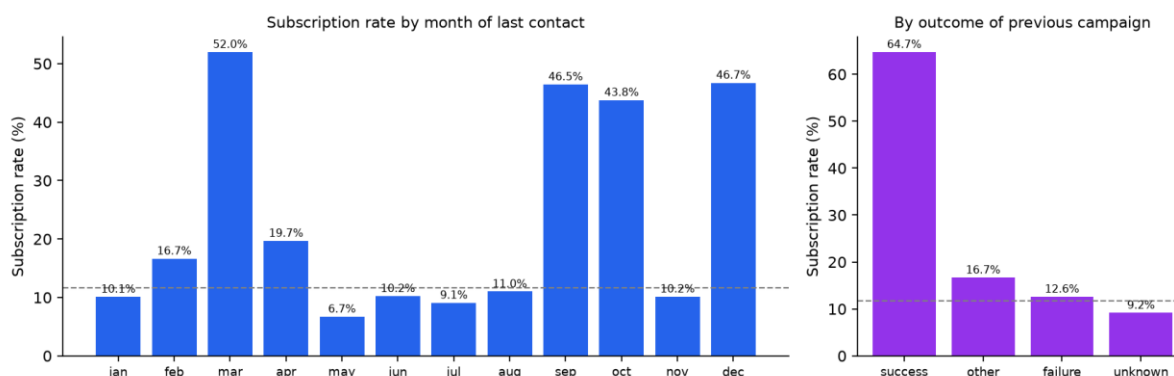


Figure 3: Subscription rate by contact month and previous campaign outcome

Timing matters strongly. May has by far the most contacts (13,766) but the lowest rate (6.7%), whereas the rarely used months of March (52.0%), September (46.5%), October (43.8%) and December (46.7%) convert at four to eight times that rate. **Past success is the strongest single signal:** clients whose previous campaign was a success subscribe at 64.7%, compared with 9.2% for clients with no previous outcome.

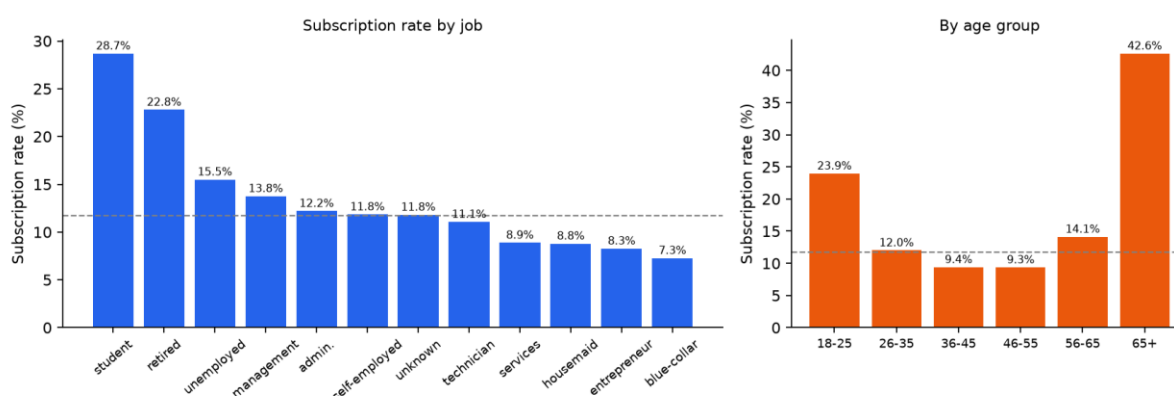


Figure 4: Subscription rate by job and age group

Demographics show a U-shape. Students (28.7%) and retired clients (22.8%) are the most receptive, blue-collar workers the least (7.3%). Similarly, clients over 65 (42.6%) and aged 18–25 (23.9%) subscribe much more often than the 36–55 group (about 9.4%). Tertiary education (15.0%) and single clients (14.9%) are also above average.

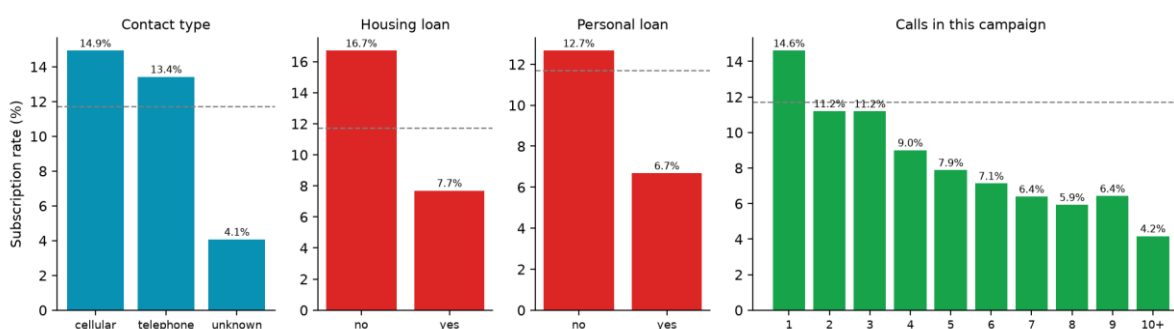


Figure 5: Subscription rate by contact type, existing loans and number of calls

Contact channel and debt. Clients reached on a mobile phone subscribe at 14.9% versus 4.1% when the channel is unknown. Clients with a housing loan (7.7%) or personal loan (6.7%) are roughly half as likely to subscribe as those without. **Repeated calls give diminishing returns:** the rate falls from 14.6% after one call to 4.2% for ten or more calls.

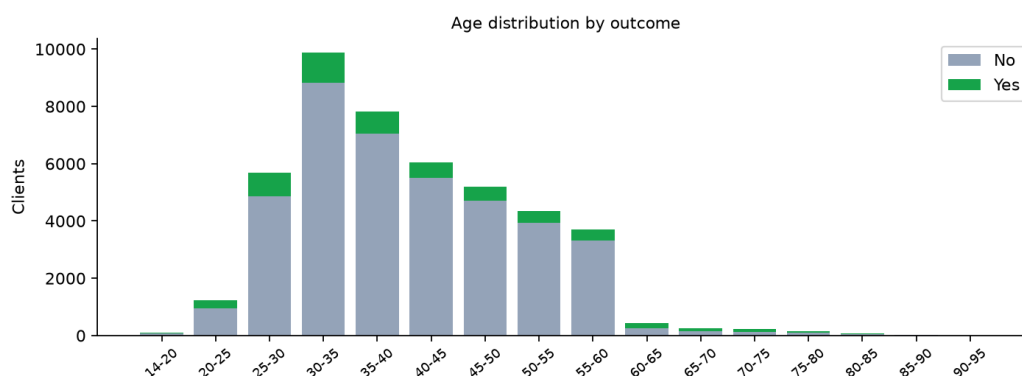


Figure 6: Age distribution split by outcome

Table 8: Linear correlation of numeric attributes with the target

Attribute	Pearson r with y
pdays	+0.104
previous	+0.093
campaign	-0.073
balance	+0.053
day	-0.028
age	+0.025

All linear correlations are weak ($|r| \leq 0.10$), which indicates that the relationships are non-linear and driven by interactions and categorical attributes. This motivated the use of tree ensembles in addition to linear models.

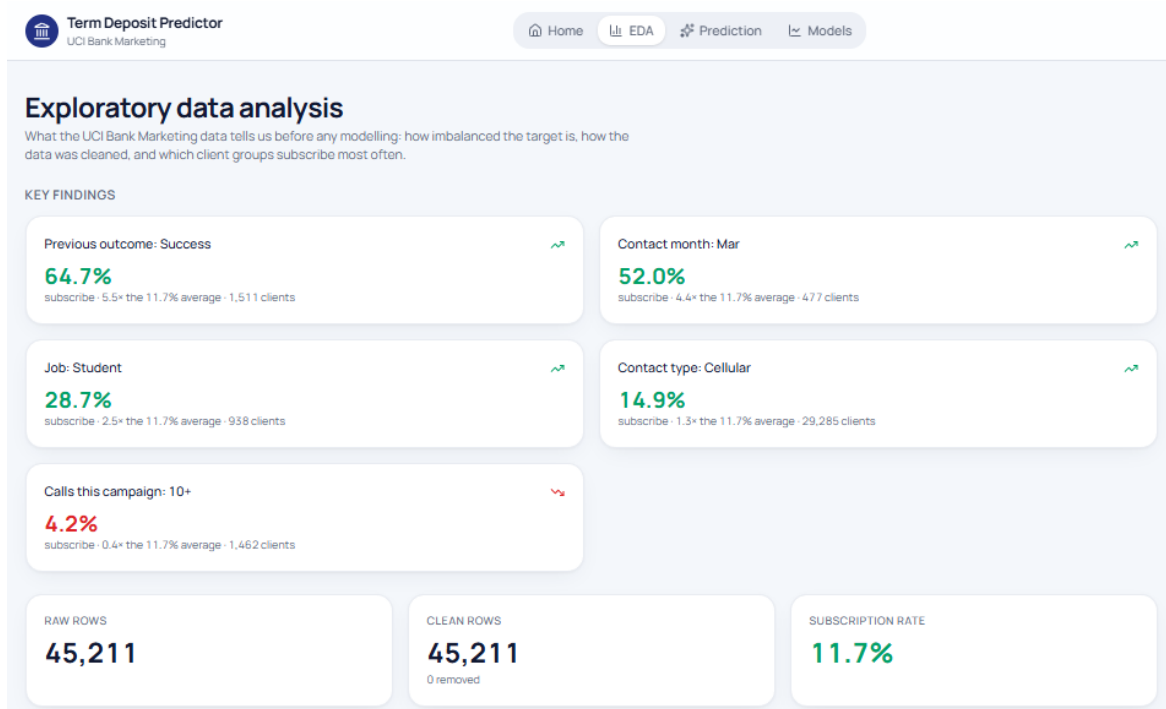


Figure 7: EDA page of the dashboard: automatically derived key findings and dataset KPIs

6. Data Mining Techniques and Modelling

6.1 Experimental design

- **Split:** stratified 80/20 train/test split (36,168 training rows, 9,043 test rows) so both sets keep the 11.7% positive rate.
- **Tuning:** `GridSearchCV` with 3-fold stratified cross-validation on the training set only, scored by ROC-AUC.
- **Selection:** the model with the highest **cross-validated** ROC-AUC is chosen, so the test set remains an unbiased estimate of real-world performance.
- **Class imbalance:** `class_weight='balanced'` (or `balanced_subsample` for Random Forest) and balanced per-row sample weights for Gradient Boosting make the rare “yes” class count as much as the “no” class.
- **Threshold:** the selected model's decision threshold is tuned to maximise F1 on out-of-fold training predictions (`cross_val_predict`), never on the test set.
- **Deployment model:** the winner is refit on all clean data with the chosen hyper-parameters and saved to `artifacts/model.joblib`.

6.2 Algorithms

Six classification algorithms from different families were compared, as summarised below.

Table 9: Classification algorithms compared

Algorithm	Family / principle	Why it was included	Search grid
Logistic Regression	Linear model; estimates log-odds as a weighted sum of features	Interpretable, fast baseline	$C \in \{0.1, 1, 10\}$
Decision Tree	Recursive partitioning on information gain (Gini)	Human-readable rules, captures interactions	$\text{max_depth} \in \{5, 8, 12\};$ $\text{min_samples_leaf} \in \{20, 50\}$

Random Forest	Bagging ensemble of 300 de-correlated trees (Breiman, 2001)	Robust, low variance, handles mixed data	max_depth ∈ {10, 16}; min_samples_leaf ∈ {5, 10}
Gradient Boosting (HistGB)	Boosting: trees fitted sequentially to residual errors on binned features (Friedman, 2001; Ke et al., 2017)	State of the art for tabular data	learning_rate ∈ {0.05, 0.1}; max_depth ∈ {4, 8}
K-Nearest Neighbours	Instance-based; majority of the k closest clients	Non-parametric, no training assumptions	k ∈ {25, 51}; weights = distance
Gaussian Naive Bayes	Probabilistic; Bayes' theorem with feature independence	Very fast probabilistic baseline	default

Table 10: Best hyper-parameters found by grid search (★ = selected)

Model	Best hyper-parameters	CV ROC-AUC	Train time (s)
Gradient Boosting ★	learning_rate=0.05, max_depth=8	0.7947	4.4
Random Forest	max_depth=16, min_samples_leaf=10	0.7869	15.2
Logistic Regression	C=0.1	0.7689	7.0
Decision Tree	max_depth=8, min_samples_leaf=50	0.7619	3.0
K-Nearest Neighbours	n_neighbors=51, weights=distance	0.7504	2.5
Naive Bayes	defaults	0.7417	0.3

7. Evaluation and Results

7.1 Evaluation metrics

Accuracy is misleading for this problem: a model that always predicts “no” is 88.3% accurate but finds no subscribers. We therefore evaluate with metrics suited to imbalanced data:

- **ROC-AUC** – probability that a random subscriber is ranked above a random non-subscriber (0.5 = random). Used for model selection because it is threshold-independent.
- **PR-AUC (average precision)** – area under the precision-recall curve; stricter under imbalance (random = 0.117).
- **Precision** – of the clients flagged “yes”, the share who actually subscribe (call efficiency).
- **Recall** – of the actual subscribers, the share the model finds (missed sales).
- **F1** – harmonic mean of precision and recall; used to choose the decision threshold.
- **Cumulative gains / lift** – share of subscribers reached when calling the top-k% of ranked clients (business view).

7.2 Model comparison

Table 11: Test-set performance of all models at the default 0.5 threshold, ranked by CV ROC-AUC

Rank	Model	CV ROC-AUC	Test ROC-AUC	PR-AUC	F1	Precision	Recall	Accuracy
1	Gradient Boosting ★	0.795	0.806	0.466	0.459	0.358	0.639	0.824
2	Random Forest	0.787	0.797	0.442	0.457	0.377	0.580	0.839
3	Logistic Regression	0.769	0.776	0.415	0.383	0.274	0.636	0.760
4	Decision Tree	0.762	0.773	0.398	0.428	0.340	0.579	0.819
5	K-Nearest Neighbours	0.750	0.767	0.396	0.208	0.674	0.123	0.890

6	Naive Bayes	0.742	0.752	0.352	0.403	0.351	0.472	0.836
---	-------------	-------	-------	-------	-------	-------	-------	-------

Gradient Boosting ranks first on every threshold-independent metric (CV ROC-AUC 0.795, test ROC-AUC 0.806, PR-AUC 0.466), followed closely by Random Forest. The two tree ensembles clearly outperform the single tree and the linear, instance-based and probabilistic models, which confirms the EDA finding that the signal is non-linear. K-Nearest Neighbours illustrates the accuracy paradox: it has the highest accuracy (0.890) but finds only 12.3% of subscribers.

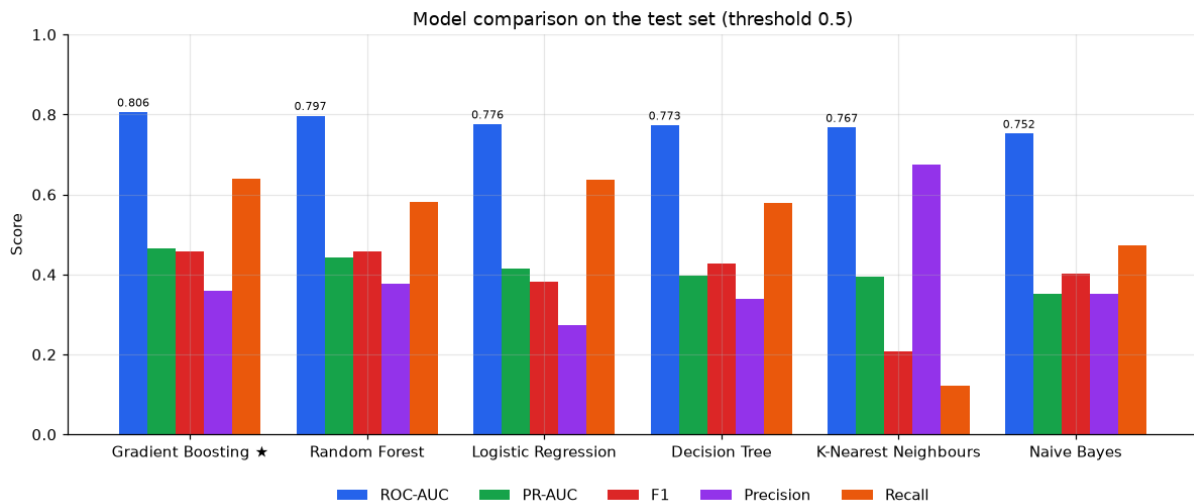


Figure 8: Test metrics of all models (threshold 0.5)

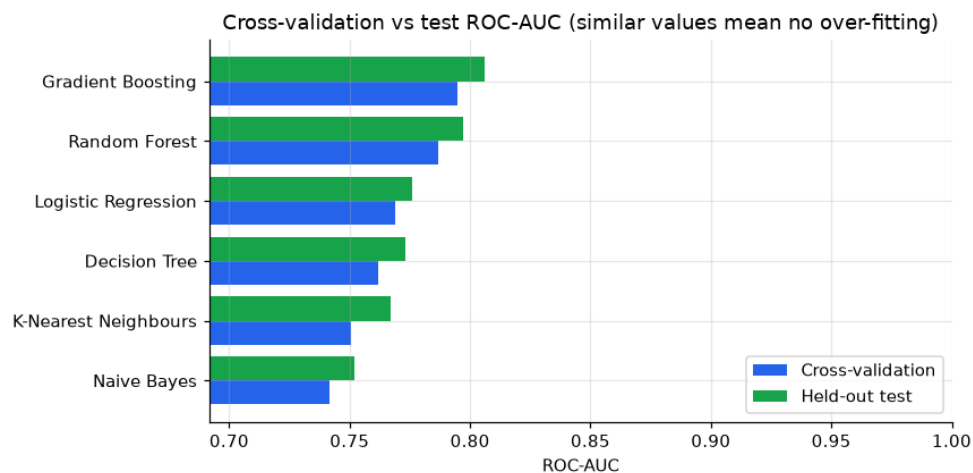


Figure 9: Cross-validation vs held-out test ROC-AUC

Cross-validation and test ROC-AUC differ by at most about 0.02 for every model, so the models generalise well and are not over-fitted to the training data.

7.3 ROC and precision-recall curves

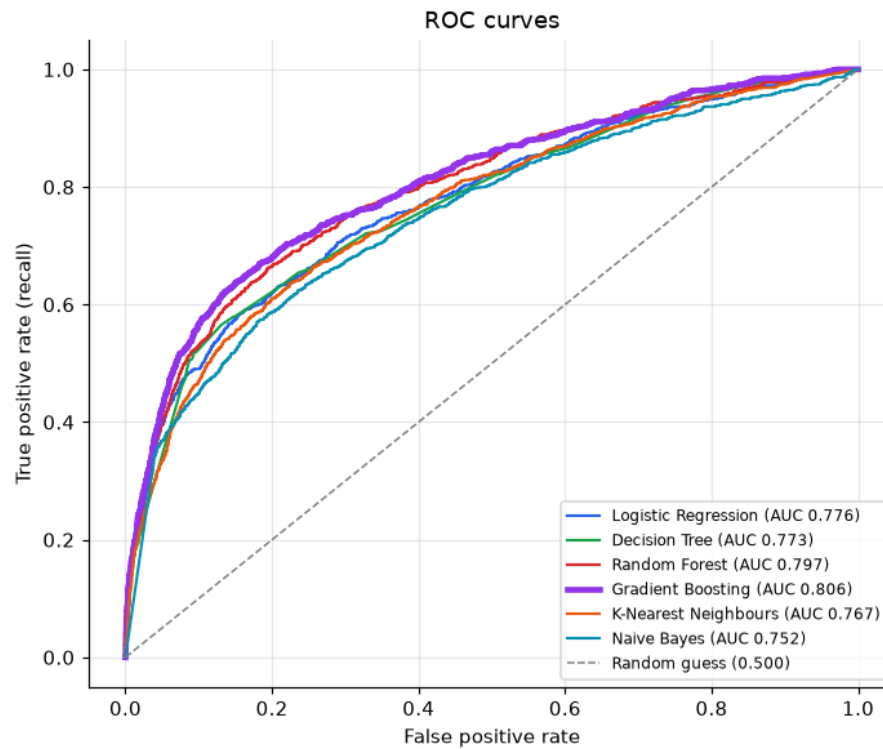


Figure 10: ROC curves of all models on the test set

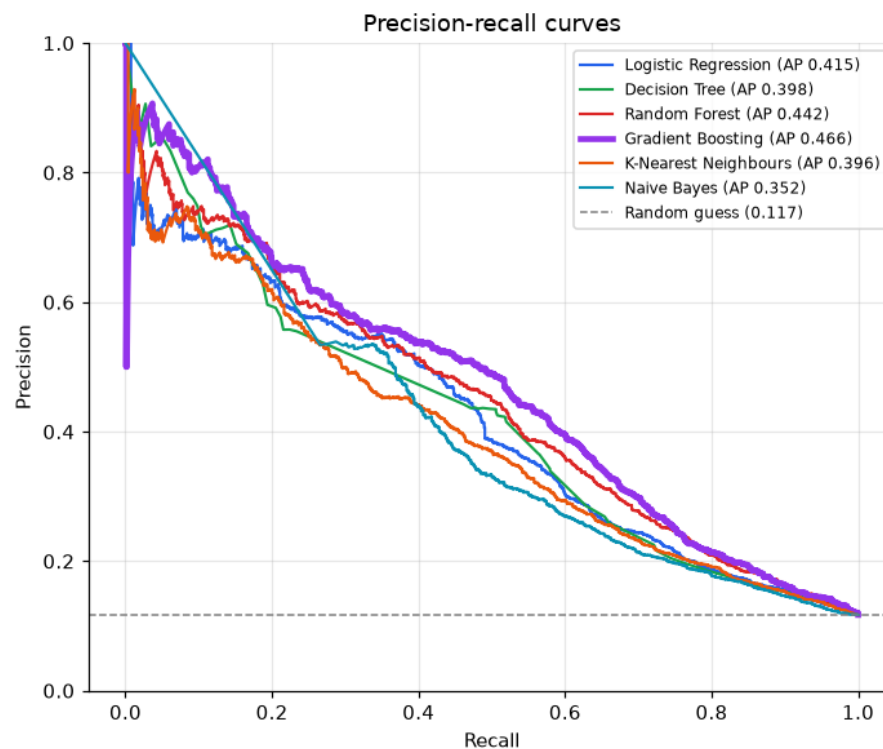


Figure 11: Precision-recall curves of all models on the test set

The selected model dominates the others across most of the ROC and PR space. Its PR-AUC of 0.466 is about 4.0 times the random baseline of 0.117.

7.4 Confusion matrices

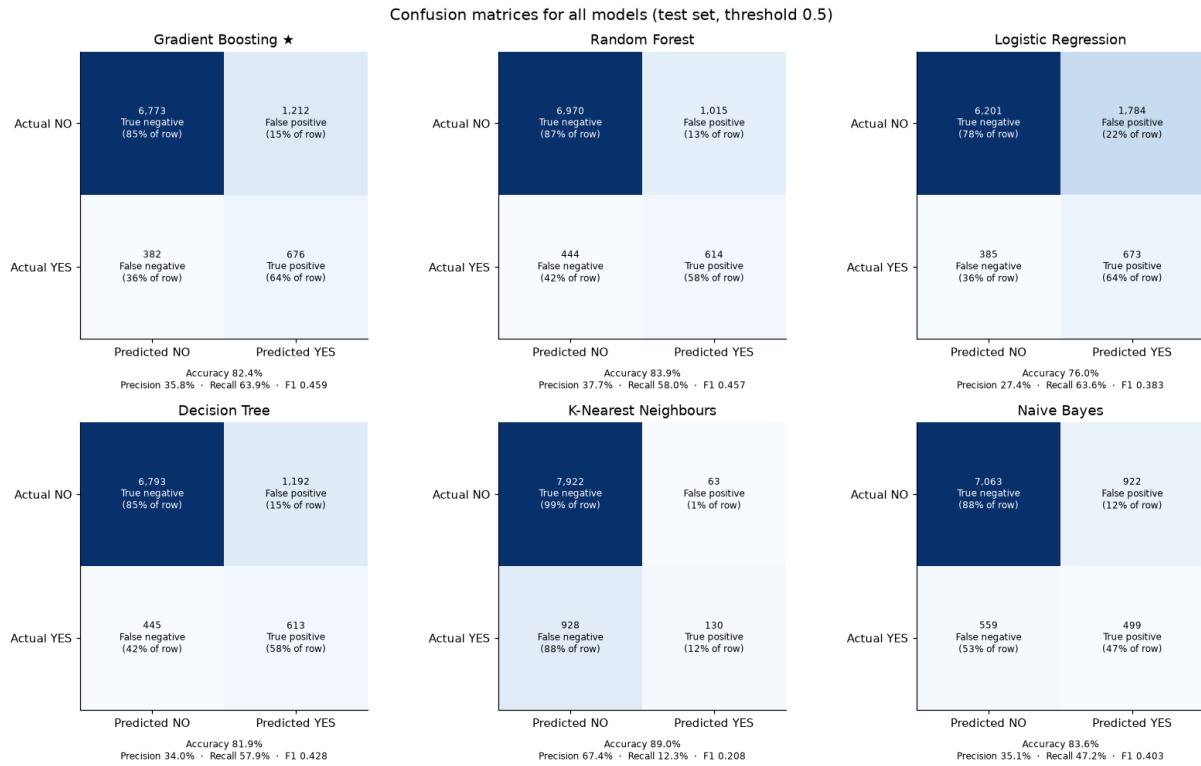


Figure 12: Confusion matrices of all models (test set, threshold 0.5)

7.5 Decision threshold tuning

Class weighting pushes scores upwards, so the default 0.5 threshold flags many clients. Tuning the threshold for F1 on out-of-fold training predictions gave **0.62**. On the test set this reduces wasted calls (false positives) from 1,212 to 690 while still finding 562 of 1,058 subscribers:

Table 12: Gradient Boosting on the test set at the default and tuned thresholds

Threshold	TN	FP (wasted call)	FN (missed)	TP	Precision	Recall	F1	Accuracy
0.50	6,773	1,212	382	676	0.358	0.639	0.459	0.824
0.62 (tuned)	7,295	690	496	562	0.449	0.531	0.487	0.869

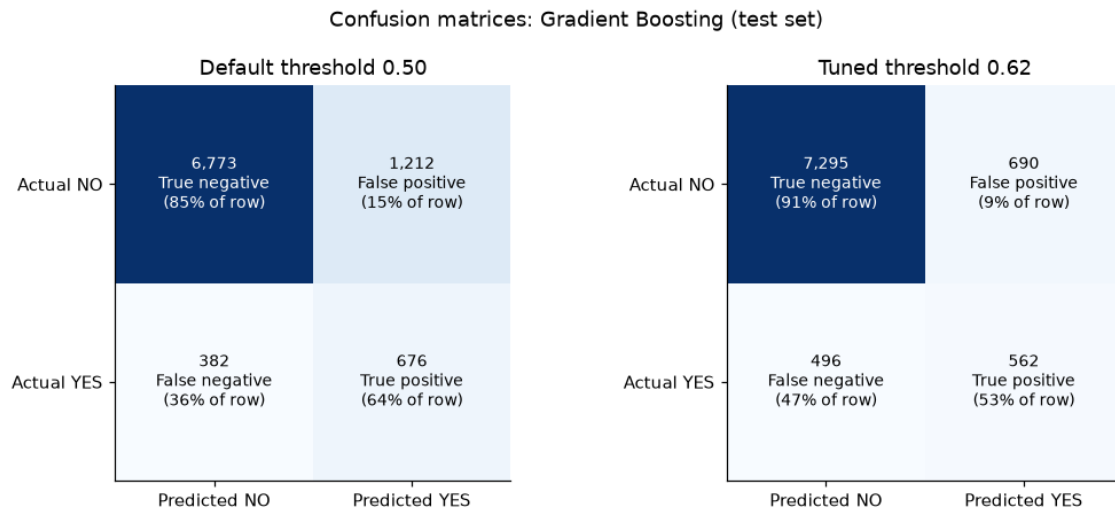


Figure 13: Confusion matrices of Gradient Boosting at the default and tuned thresholds

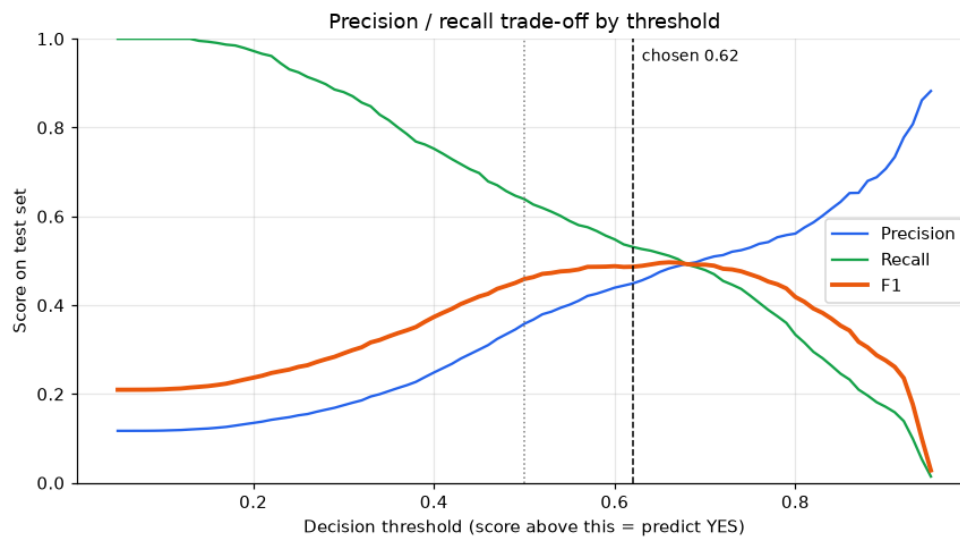


Figure 14: Precision, recall and F1 as the decision threshold changes

The threshold is a business lever: a bank with spare call capacity can lower it to increase recall, while a bank with limited agents can raise it to increase precision.

7.6 Feature importance

Permutation importance measures how much test ROC-AUC drops when the values of one raw input are randomly shuffled (5 repeats on a 4,000-row sample).

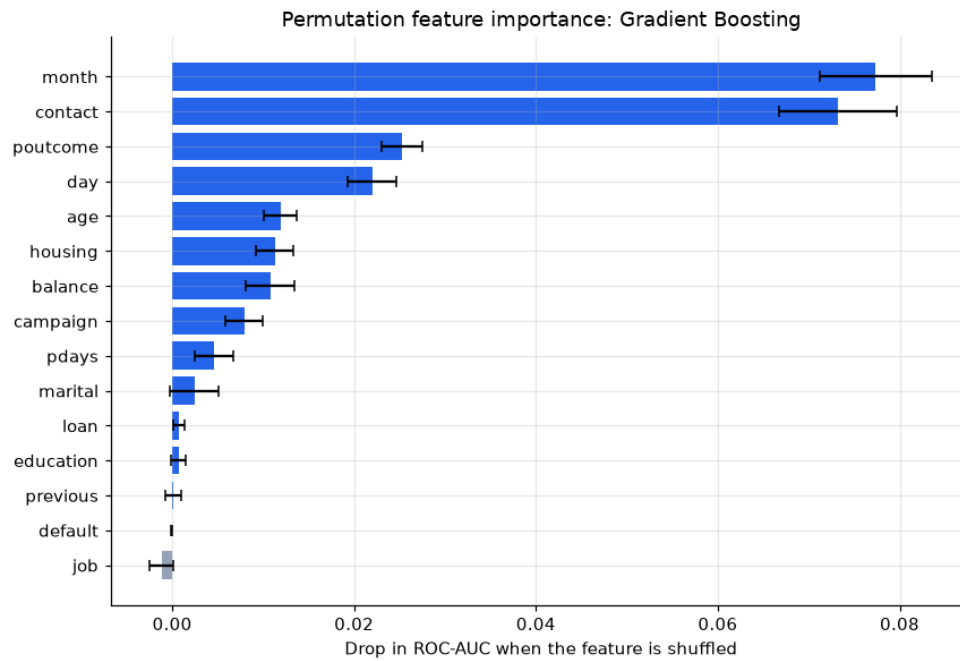


Figure 15: Permutation feature importance of Gradient Boosting

The five most important inputs are **month** (0.077), **contact** (0.073), **poutcome** (0.025), **day** (0.022), **age** (0.012). This agrees with the EDA: campaign timing, contact channel and previous outcome dominate, while **job**, **default**, **previous** and **education** add almost nothing once the other features are known.

7.7 Business evaluation: cumulative gains

Table 13: Cumulative gains of the selected model on the test set

Top % of clients called	Subscribers reached	Lift vs random
5%	26.4%	5.27x
10%	44.4%	4.44x
20%	62.8%	3.14x
30%	71.7%	2.39x
40%	77.5%	1.94x
50%	84.1%	1.68x

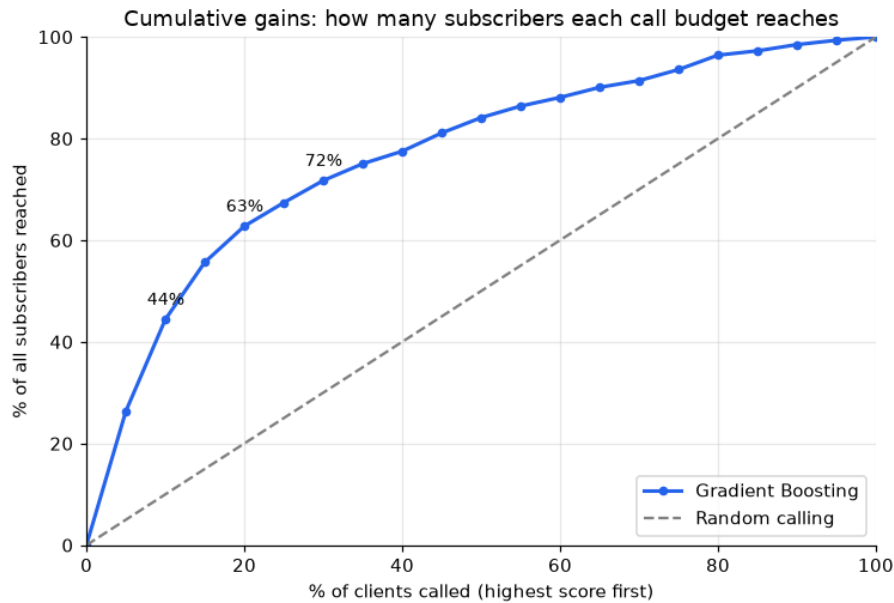


Figure 16: Cumulative gains curve vs random calling

If the bank calls clients in descending order of model score, calling only the top 20% of clients reaches **62.8%** of all subscribers, and half the call list reaches 84.1%. In other words, the same number of sales could be achieved with far fewer calls than random dialling.

8. System Design and Implementation

8.1 Architecture

The system has three layers that communicate through files and HTTP/JSON: an offline ML pipeline that produces artifacts, a stateless REST API that loads those artifacts, and a multi-page web dashboard.

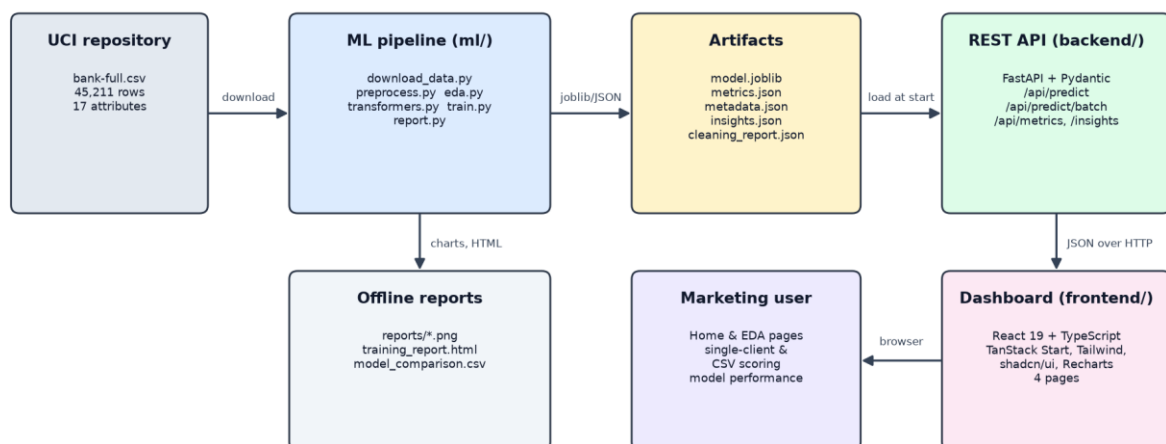


Figure 17: System architecture

8.2 Technology stack

Table 14: Technology stack

Layer	Technologies	Purpose
Data mining	Python 3.11+, pandas, NumPy, scikit-learn, joblib	Cleaning, EDA, pipelines, model training and persistence

Reporting	matplotlib, HTML	Charts and a self-contained training report (reports/)
Backend	FastAPI, Pydantic v2, Uvicorn	REST API, input validation, OpenAPI docs
Frontend	React 19, TypeScript, TanStack Start (Router + Query), Vite, Tailwind CSS v4, shadcn/ui (Radix UI), Recharts, lucide-react, sonner	Routed multi-page dashboard, data fetching and caching, charts and UI components
Testing	pytest, FastAPI TestClient (httpx); Vitest, Testing Library, jsdom	Automated API and frontend tests
Code quality	ESLint, Prettier, TypeScript strict typing	Linting, consistent formatting and static type checks
Version control	Git / GitHub	Collaboration and history

8.3 Repository structure

FDM-Bank_Marketing/	
ml/	data mining pipeline
config.py	paths, feature lists, random seed
download_data.py	fetch UCI archive, extract bank-full.csv
preprocess.py	cleaning + cleaning_report.json
eda.py	exploratory aggregates -> insights.json
transformers.py	FeatureEngineer, QuantileCapper
train.py	tune/compare 6 models, threshold, importance
report.py	charts, HTML report, comparison CSV
run_pipeline.py	runs everything in order
artifacts/	model.joblib + JSON consumed by the API
reports/	PNG charts, training_report.html
backend/app/	FastAPI app (main.py, model_service.py, schemas.py)
backend/tests/	pytest API tests
frontend/	React dashboard (TanStack Start + Vite)
vite.config.ts	dev server :5173, /api proxy to :8000
src/	
routes/	__root, index (Home), eda, predict, models
components/	
layout/	AppShell: header, navigation, footer
dashboard/	PredictPage, BatchPage, PerformancePage, InsightsPage
ui/	shadcn/ui components (Radix UI)
lib/	api.ts (client + demo data), queries.ts, fields.ts
test/	Vitest tests

8.4 Backend REST API

The API ([backend/app/main.py](#)) loads the model and JSON artifacts once at start-up through a [ModelService](#) singleton. Requests are validated by Pydantic schemas ([schemas.py](#)) that restrict every categorical field to its allowed values and every numeric field to a sensible range, so invalid input is rejected with HTTP 422 before reaching the model.

Table 15: REST API endpoints

Method	Endpoint	Description
GET	/api/health	Service status and loaded model name
GET	/api/metadata	Input schema (allowed categories, ranges, defaults), threshold, cleaning steps
GET	/api/metrics	All model results, ROC curves, confusion matrix, feature importance
GET	/api/insights	EDA aggregates and the cleaning report
POST	/api/predict	Score one client (JSON body with the 15 input fields)
POST	/api/predict/batch	Score an uploaded CSV (comma or semicolon separated)

Each prediction returns the decision, the score, the threshold and a lead-priority band: **high** (score $\geq$ threshold), **medium** ($\geq$ half the threshold) or **low**. The batch endpoint auto-detects the delimiter, ignores extra columns such as **y** or **duration** (so the original UCI file can be uploaded directly), validates each row independently, scores all valid rows in one vectorised call and reports invalid rows individually. Uploads are limited to 10 MB and 50,000 rows.

```
POST /api/predict
{"age": 35, "job": "management", "marital": "married", "education": "tertiary",
 "default": "no", "balance": 1500, "housing": "yes", "loan": "no", "contact": "cellular",
 "day": 15, "month": "may", "campaign": 2, "pdays": -1, "previous": 0, "poutcome": "unknown"}
-> {"prediction": "no", "probability": 0.4659, "threshold": 0.62, "likelihood": "medium"}
```

Bank Marketing Term Deposit Predictor 1.0.0 OAS 3.1

/openapi.json

Predicts whether a client will subscribe to a term deposit (UCI Bank Marketing dataset).

default

GET

/api/health Health

GET

/api/metadata Metadata

GET

/api/metrics Metrics

GET

/api/insights Insights

POST

/api/predict Predict

POST

/api/predict/batch Predict Batch

Schemas

BatchResponse >

Expand all object

BatchRowError >

Expand all object

Figure 18: Auto-generated interactive API documentation (/docs)

8.5 Frontend dashboard

The dashboard (**frontend/**) is a React 19 + TypeScript application built with TanStack Start and Vite. Each page is a file-based route in **src/routes/**, server data is fetched and cached with TanStack Query, and the interface uses Tailwind CSS, shadcn/ui (Radix UI) components, Recharts and lucide icons. The prediction form is generated from **/api/metadata**, so dropdown options, defaults and limits always match the trained model. A shared AppShell provides the sticky header with navigation, a pill showing the serving model and its ROC-AUC, and a footer crediting the data source. The dashboard has four pages:

- **Home (/)** – introduces the problem and shows a “Serving model” card (Gradient Boosting: ROC-AUC 80.6%, recall 53.1%, precision 44.9%, threshold 0.62), KPI cards (45,211 calls analysed, 11.7% subscription rate, 6 models compared, 15 model inputs), a five-step “How it works” timeline (data, cleaning, modelling, threshold, serving) and the inputs the model uses.

- **EDA (/eda)** – key-finding cards computed from the insights data (previous success 64.7%, March 52.0%, students 28.7%, cellular 14.9%, ten or more calls 4.2%), raw and clean row counts, class balance, cleaning steps, subscription rate by any attribute, age distribution, effect of campaign calls and correlations.
- **Prediction (/predict)** – two tabs. **Single client**: a form grouped into client profile, finances, current campaign and previous campaign, with switches for yes/no fields; it shows a YES/NO verdict, the probability bar with the threshold marker and a likelihood badge (lead priority). **Batch (CSV)**: drag-and-drop upload (max 10 MB), KPI cards (total, yes, no, failed rows), a Yes/No filter, sortable and paginated results, a row-error panel and export to `*_predictions.csv`.
- **Models (/models)** – comparison table of the six models with the best one highlighted, grouped test-score chart, ROC curves, confusion matrix at the tuned threshold and permutation feature importance with error bars.

API errors such as validation failures are shown as toast notifications. If the backend cannot be reached, the API client (`lib/api.ts`) falls back to built-in sample data and the header shows a “Demo data” badge, so the interface can still be demonstrated offline; all results in this report come from the live API.

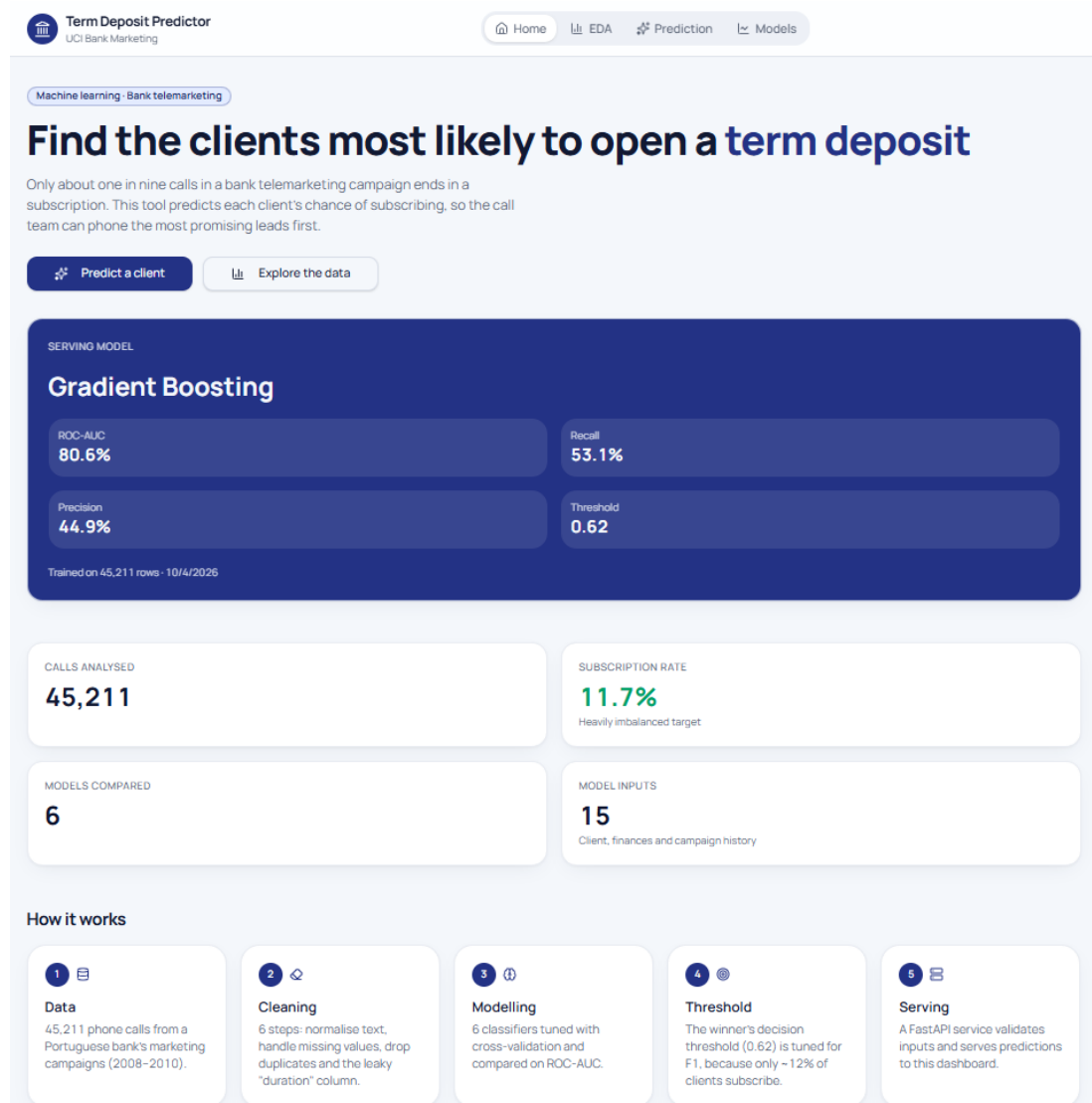


Figure 19: Home page: serving model, dataset KPIs and how the system works

Term Deposit Predictor
UC Bank Marketing

Home
EDA
Prediction
Models

Prediction

Estimate how likely a client is to subscribe to a term deposit. Score a single client with the form, or upload a CSV to score a whole call list.

Single client
Batch (CSV)

Client profile

Who the client is

Age

18 - 100

Job

Marital status

divorced includes widowed

Education

Finances

Credit and balance

Credit in default

☐ No

Average yearly balance (€)

100000 - 1,000,000

Housing loan

☒ Yes

Personal loan

☐ No

Current campaign

Last contact in this campaign

Contact type

Day of month

1 - 31

Month

Contacts this campaign

including last contact

Previous campaign

History with earlier campaigns

Days since last contact

-1 = never contacted

Previous contacts

0 - 300

Previous outcome

Prediction

Will this client subscribe to a term deposit?

NO

Unlikely to subscribe

Probability

38.6%

0% threshold 62.0% 100%

Likelihood

Medium

Predict

Reset to defaults

Figure 20: Prediction page, Single client tab: the default client scores 38.6%, below the 0.62 threshold (medium priority)



Term Deposit Predictor
UCI Bank Marketing

[Home](#)
[EDA](#)
[Prediction](#)
[Models](#)

Prediction

Estimate how likely a client is to subscribe to a term deposit. Score a single client with the form, or upload a CSV to score a whole call list.

☒ Single client
☒ Batch (CSV)



Drop a CSV here or click to browse

Max 10 MB · comma or semicolon separated · same 15 columns as the form

No file scored yet

Upload a CSV to score many clients at once

Data: UCI Machine Learning Repository, Bank Marketing (Moro et al., 2014)
FastAPI · scikit-learn · React

Figure 21: Prediction page, Batch (CSV) tab ready for an upload



Term Deposit Predictor
UCI Bank Marketing

[Home](#)
[EDA](#)
[Prediction](#)
[Models](#)

Model performance

Six classifiers tuned with cross-validation and scored on a held-out test set. The best model by ROC-AUC is the one serving predictions.

Model comparison

Held-out test set scores. Best model highlighted.

Model	CV ROC-AUC	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Train time
Logistic Regression	76.9%	76.0%	27.4%	63.6%	38.3%	77.6%	41.5%	7.0s
Decision Tree	76.2%	81.9%	34.0%	57.9%	42.8%	77.3%	39.8%	3.0s
Random Forest	78.7%	83.9%	37.7%	58.0%	45.7%	79.7%	44.2%	15.2s
Gradient Boosting Best	79.5%	82.4%	35.8%	63.9%	45.9%	80.6%	46.6%	4.4s
K-Nearest Neighbours	75.0%	89.0%	67.4%	12.3%	20.8%	76.7%	39.6%	2.5s
Naive Bayes	74.2%	83.6%	35.1%	47.2%	40.3%	75.2%	35.2%	0.3s

Figure 22: Models page: comparison of the six models on the held-out test set

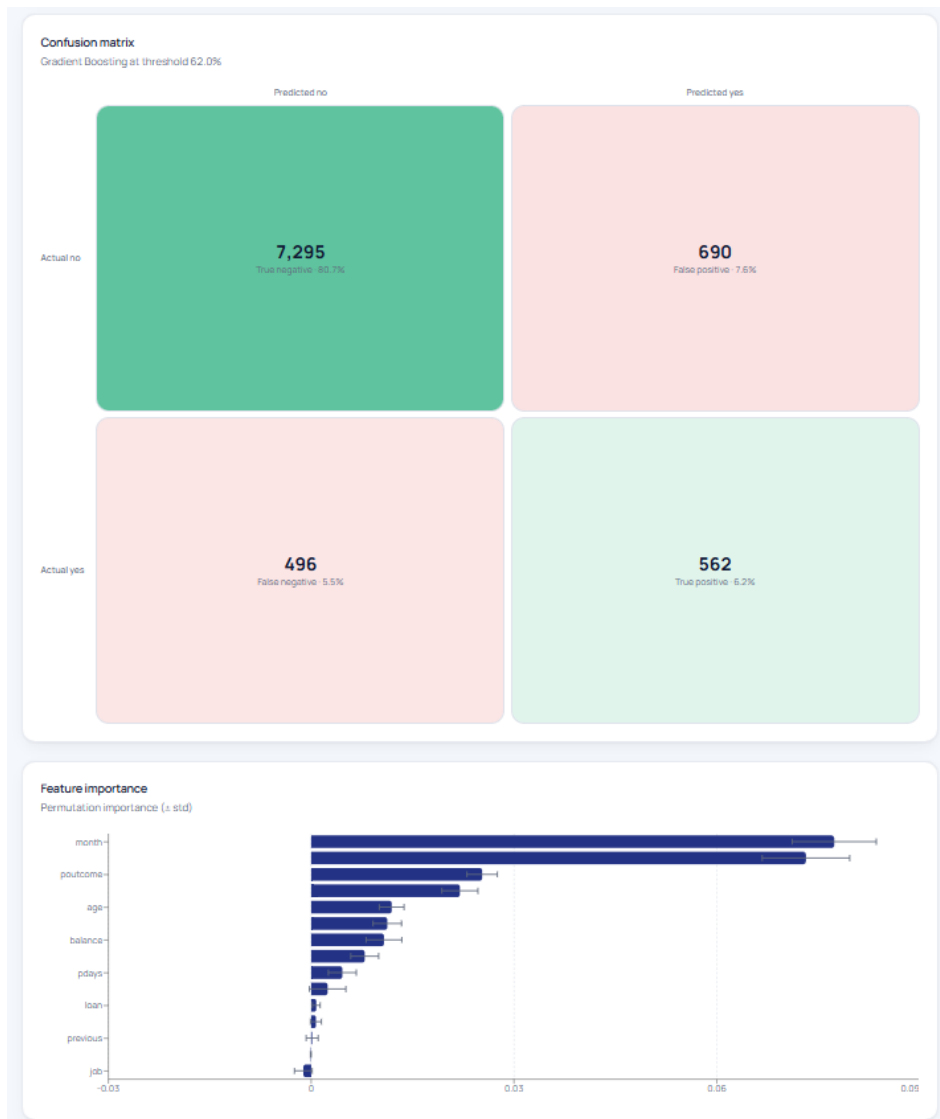


Figure 23: Models page: confusion matrix at the tuned threshold and permutation feature importance

9. Testing and Validation

Automated tests cover both tiers. The API tests in `backend/tests/test_api.py` run in-process with FastAPI's `TestClient` and the real trained model (`pytest`: 8 passed). The frontend uses Vitest with Testing Library in a jsdom environment (`frontend/src/test/`); its routing test checks the generated route tree (`npm test`: 1 passed). All 9 tests pass.

Table 16: Automated test cases

#	Test	What it verifies	Result
1	test_health	API is up and the model is loaded	Pass
2	test_metadata_and_metrics	Metadata lists categories; metrics contain all 6 models and agree on the selected model	Pass
3	test_insights	EDA covers the full dataset (> 40,000 rows) and the cleaning report is present	Pass
4	test_predict_single	A valid record returns a score in [0, 1] and a yes/no decision	Pass
5	test_previous_success_raises_probability	Domain sanity check: a previous campaign success raises the score	Pass
6	test_predict_validation_error	Out-of-range age and unknown job are rejected with HTTP 422	Pass

7	test_batch	A CSV with one valid and one invalid row scores the valid row and reports row 2 as an error	Pass
8	test_batch_missing_columns	A CSV without the required columns is rejected with HTTP 400	Pass
9	app-routing (Vitest)	The file-based route tree resolves / to the Home page instead of the not-found fallback	Pass

Model validity was assessed separately through stratified cross-validation, a held-out test set that was never used for fitting, selection or threshold tuning, and the comparison of CV and test scores (Section 7.2). The dashboard was tested manually end-to-end with the backend running, including navigation between the four pages, single-client prediction, invalid input messages, CSV upload of the original bank-full.csv, filtering, sorting and CSV export.

10. Deployment and User Guide

Prerequisites: Python 3.11+ and Node.js 20.19+ (required by Vite 8). The training pipeline takes about 1–2 minutes on a laptop.

```
# 1. Python environment
python -m venv .venv
.\.venv\Scripts\activate          # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt

# 2. Download data, clean, EDA, train and generate reports
python -m ml.run_pipeline

# 3. Backend API on http://127.0.0.1:8000 (docs at /docs)
cd backend
uvicorn app.main:app --port 8000

# 4. Frontend on http://localhost:5173 (new terminal)
cd frontend
npm install
npm run dev                      # proxies /api to http://127.0.0.1:8000

# Tests and checks
cd backend && pytest
cd frontend && npm test && npm run lint

# Production build of the dashboard
cd frontend && npm run build
```

The Vite development server proxies `/api` requests to the backend, so both processes must be running; set `VITE_API_URL` to call an API hosted elsewhere instead. To retrain on new campaign data, replace the raw CSV and re-run the pipeline; the API picks up the new artifacts on restart and the dashboard adapts automatically through `/api/metadata`.

11. Discussion

11.1 Business insights and recommendations

9. **Prioritise calls by model score.** Calling the top 20% of ranked clients reaches 63% of subscribers.
10. **Re-target past successes.** Clients with a previous successful campaign convert at 65%.
11. **Rethink campaign timing.** May is heavily used but converts at only 6.7%; March, September, October and December convert above 40% and deserve more capacity.

12. **Prefer mobile contact** and record the contact channel: unknown-channel contacts convert at only 4.1%.
13. **Cap repeated calls.** Conversion drops from 14.6% on the first call to 4.2% after ten; further calls mostly waste effort and risk annoying clients.
14. **Target receptive segments** such as students, retired clients and clients without existing loans.

11.2 Limitations

- Precision at the tuned threshold is about 45%: more than half of flagged clients still do not subscribe. Without **duration**, the available pre-call attributes contain limited signal.
- Because training uses class weights, the output score ranks clients but is not a calibrated probability.
- The data covers 2008–2010 during the financial crisis; macro-economic conditions have changed, so the model should be retrained on recent data before real use.
- Month and day effects may partly reflect how the bank scheduled campaigns rather than client behaviour.
- The prototype has no authentication, audit logging or production hosting.
- The dashboard's offline demo data is illustrative and differs from the trained model (for example a 0.32 threshold and four models). It is clearly badged, but decisions should only be based on the live API.

11.3 Future work

- Add probability calibration (Platt scaling or isotonic regression) so scores can be read as probabilities.
- Use cost-sensitive threshold selection based on the bank's actual cost per call and profit per deposit.
- Try additional algorithms (XGBoost, LightGBM, CatBoost) and the **bank-additional** socio-economic indicators.
- Add SHAP explanations for individual predictions in the dashboard.
- Containerise with Docker, add authentication, and deploy to the cloud with CI/CD and model monitoring for drift.

12. Individual Contributions

All four members contributed to every stage of this project, rather than one member owning a single stage outright. Each of the four stages below (data understanding and preprocessing, exploratory data analysis and feature engineering, model development and evaluation, and system implementation and testing) was carried out jointly, with every member doing identifiable work in each stage. The specific task each member performed for that stage is set out **below each stage heading**. Section 12.5 additionally records, for reference, which files each member primarily worked on within this shared effort.

Table 17: Summary of contribution by project stage

Project stage	Work covered	Report sections
Data understanding, cleaning & validation	Data acquisition, quality assessment, cleaning, domain-rule validation, removal of the leaking duration attribute and target encoding.	2, 4.1–4.2, 10

Exploratory data analysis & feature engineering	Subscription-rate analysis by category, outlier investigation, feature engineering (previously_contacted, log_balance, total_contacts, age_group) and outlier capping.	4.3–4.4, 5, 11.1
Model training, tuning & evaluation	Comparison of six classifiers, stratified cross-validation, hyper-parameter tuning, F1-based threshold tuning and permutation feature importance.	3, 6, 7
System implementation & testing	FastAPI backend (prediction, batch scoring, metrics endpoints), React dashboard (Home, EDA, Prediction and Models pages) and the automated test suites (pytest, Vitest).	1, 8, 9, 11.2–11.3

All members jointly took part in problem definition, choosing the dataset, weekly progress meetings, code reviews, integration testing, proofreading this report and preparing the presentation.

12.1 Data Acquisition, Cleaning and Validation

Member	What they did for this stage
Gunathilaka W.A.D.S.	Wrote the data-download script and the cleaning/validation logic (ml/download_data.py, ml/preprocess.py, ml/config.py); produced cleaning_report.json.
Edirisinghe E.M.K.L.	Reviewed the cleaning rules and cross-checked the quality report against the raw dataset for consistency.
Illesinghe E.S.	Tested the preprocessing pipeline against edge cases (e.g. the "unknown" categories and the pdays sentinel value).
Kaluarachchi R.G.	Documented the dataset description, data-quality observations and limitations sections of this report.

- Implemented `ml/download_data.py`, which downloads the UCI archive and recursively searches the nested ZIP files to extract `bank-full.csv`, skipping the download when the file already exists.
- Created `ml/config.py`, the single source of truth for folder paths, artifact locations, the raw input columns, engineered columns and the random seed.
- Implemented the cleaning stage in `ml/preprocess.py`: text normalisation, missing-value analysis and imputation strategy, duplicate removal, domain-rule validation, removal of the leaking `duration` attribute and target encoding, with every step recorded in `cleaning_report.json`.
- Analysed the “unknown” placeholders and justified keeping them as a category based on their distinct subscription rates.
- Built the batch-scoring feature: the `/api/predict/batch` endpoint (delimiter detection, header normalisation, per-row validation and error reporting, upload limits) and the Batch (CSV) tab of the Prediction page (`BatchPage.tsx`) with drag-and-drop upload, file checks, KPI cards, Yes/No filtering, sorting, pagination, a row-error panel and CSV export.
- Wrote the batch-scoring test cases and the dataset description, data cleaning, leakage and deployment sections of this report.

Main files: `ml/download_data.py`, `ml/config.py`, `ml/preprocess.py`,
`artifacts/cleaning_report.json`, `backend/app/main.py` (batch endpoint),
`frontend/src/components/dashboard/BatchPage.tsx`

12.2 Exploratory Data Analysis and Feature Engineering

Member	What they did for this stage
Edirisinghe E.M.K.L.	Implemented the EDA script and the feature-engineering transformers (ml/eda.py, ml/transformers.py); built the EDA dashboard page with its key-finding cards (routes/eda.tsx, InsightsPage.tsx).

Illesinghe E.S.	Verified the feature-engineering logic and reproduced the EDA charts independently to confirm the findings.
Kaluarachchi R.G.	Cross-checked the outlier-capping thresholds and reviewed the EDA page for correctness.
Gunathilaka W.A.D.S.	Proofread the EDA findings and feature-engineering write-up in this report.

- Implemented `ml/eda.py`, which computes subscription rates per category (job, marital, education, contact, previous outcome, housing, loan, month, age group), the age histogram, the campaign-call effect, numeric correlations and summary statistics, saved to `insights.json`.
- Interpreted the EDA results and identified the key drivers (timing, previous outcome, contact channel, age U-shape, loans, diminishing returns of repeated calls) that guided feature engineering and model choice.
- Designed and implemented the custom transformers in `ml/transformers.py`: `FeatureEngineer` (previously_contacted, days_since_prev, log_balance, total_contacts, age_group, removal of `pdays`) and `QuantileCapper` for train-only winsorisation of outliers.
- Built the EDA page (`routes/eda.tsx`, `InsightsPage.tsx`) with automatically derived key-finding cards (best category per attribute among groups of at least 200 clients, compared with the 11.7% average), the class-balance chart, cleaning steps, attribute selector for subscription rates, age distribution, campaign-call chart and correlations.
- Produced the EDA figures and wrote the feature engineering, outlier handling, EDA and business recommendations sections of this report.

Main files: `ml/eda.py`, `ml/transformers.py`, `artifacts/insights.json`, `frontend/src/routes/eda.tsx`, `frontend/src/components/dashboard/InsightsPage.tsx`, `docs/figures/eda_*.png`

12.3 Model Training, Tuning and Evaluation

Member	What they did for this stage
Illesinghe E.S.	Implemented the model-training pipeline, hyper-parameter grids and evaluation code (<code>ml/train.py</code> , <code>ml/report.py</code>); selected and tuned the final model.
Kaluarachchi R.G.	Reviewed the hyper-parameter grids and verified the <code>metrics.json</code> outputs against the printed training log.
Gunathilaka W.A.D.S.	Tested the training pipeline end-to-end and confirmed the results were reproducible on a fresh run.
Edirisinghe E.M.K.L.	Proofread the methodology and evaluation sections of this report.

- Implemented `ml/train.py`: the leakage-free preprocessing + model pipeline, the stratified 80/20 split, the six candidate models and their hyper-parameter grids, and grid search with 3-fold stratified cross-validation scored by ROC-AUC.
- Handled class imbalance with balanced class weights and balanced sample weights for Gradient Boosting.
- Implemented model selection by CV ROC-AUC, F1-based threshold tuning on out-of-fold predictions, permutation feature importance and refitting the final model on all data.
- Implemented `ml/report.py`, which prints the comparison tables and produces the ROC, precision-recall, confusion-matrix, threshold, feature-importance and cumulative-gains charts and the HTML training report.

- Built the Models page (`routes/models.tsx`, `PerformancePage.tsx`) with the comparison table highlighting the best model, grouped test-score chart, ROC curves, confusion matrix at the tuned threshold and permutation importance with error bars.
- Wrote the methodology, data mining techniques and evaluation sections of this report.

Main files: `ml/train.py`, `ml/report.py`, `ml/run_pipeline.py`, `artifacts/metrics.json`, `reports/`, `frontend/src/routes/models.tsx`, `frontend/src/components/dashboard/PerformancePage.tsx`

12.4 System Implementation (Backend & Frontend) and Testing

Member	What they did for this stage
Kaluarachchi R.G.	Implemented the FastAPI backend and the frontend app shell, routing, Home page and Single client prediction (<code>main.py</code> , <code>model_service.py</code> , <code>schemas.py</code> , <code>AppShell.tsx</code> , <code>__root.tsx</code> , <code>index.tsx</code> , <code>PredictPage.tsx</code>); wrote the automated pytest and Vitest tests.
Gunathilaka W.A.D.S.	Tested the REST API endpoints and verified the batch-scoring integration end-to-end.
Edirisinghe E.M.K.L.	Reviewed the four dashboard pages (Home, EDA, Prediction, Models) against the insights requirements.
Illesinghe E.S.	Wrote the testing and deployment sections of this report.

- Designed the overall system architecture and the API contract between the ML pipeline, backend and frontend.
- Implemented the FastAPI application (`main.py`): start-up model loading, CORS, health, metadata, metrics, insights and single-prediction endpoints, and HTTP 503 handling when the model is not trained.
- Implemented `model_service.py` (artifact loading, prediction, tuned threshold and lead-priority bands) and the Pydantic request/response schemas with strict category and range validation (`schemas.py`).
- Built the application shell and data layer: the root route and Home page (`__root.tsx`, `index.tsx`), the Prediction route (`predict.tsx`), the AppShell header and navigation, the typed API client with error toasts and demo-data fallback (`api.ts`), the TanStack Query hooks (`queries.ts`) and the field definitions (`fields.ts`), plus the Single client form (`PredictPage.tsx`) with the probability bar, threshold marker and likelihood badge.
- Wrote the automated pytest suite (health, metadata/metrics, insights, single prediction, domain sanity check, validation errors) and the Vitest routing test, and carried out end-to-end integration testing.
- Wrote the introduction, system design and implementation, testing, limitations and future work sections of this report.

Main files: `backend/app/main.py`, `backend/app/model_service.py`, `backend/app/schemas.py`, `backend/tests/test_api.py`, `frontend/src/routes/` (`__root`, `index`, `predict`), `frontend/src/components/layout/AppShell.tsx`, `frontend/src/components/dashboard/PredictPage.tsx`, `frontend/src/lib/` (`api`, `queries`, `fields`), `frontend/src/test/`

12.5 Individual Task Allocation (For Reference)

Contribution was equal (25% each) across every stage described above. The table below additionally records, purely for traceability and in response to the requirement to state what each member did, which specific files each member primarily implemented within that shared, jointly-reviewed effort.

Table 17b: Individual task allocation by member

Member	Student ID	Primarily worked on (within the shared, equally-contributed effort)
Gunathilaka W.A.D.S.	IT23859456	ml/download_data.py, ml/config.py, ml/preprocess.py (data acquisition, cleaning, validation); the /api/predict/batch endpoint and BatchPage.tsx (Batch (CSV) tab of the Prediction page).
Edirisinghe E.M.K.L.	IT23857780	ml/eda.py, ml/transformers.py (EDA, feature engineering, outlier capping); routes/eda.tsx and InsightsPage.tsx (EDA page).
Illesinghe E.S.	IT23557024	ml/train.py, ml/report.py, ml/run_pipeline.py (model training, tuning, evaluation); routes/models.tsx and PerformancePage.tsx (Models page).
Kaluarachchi R.G.	IT23544918	backend/app/main.py, model_service.py, schemas.py (REST API); frontend routes (__root, index, predict), AppShell.tsx, api.ts, queries.ts, fields.ts, PredictPage.tsx; automated pytest and Vitest tests.

13. Conclusion

We developed an end-to-end data mining system that predicts term deposit subscription from information available before a telemarketing call. A careful preparation process (validation, leakage removal, feature engineering and a pipeline that is fitted only on training data) and a fair comparison of six algorithms identified Gradient Boosting as the best model, with a test ROC-AUC of 0.806 and a 4.4x lift in the top decile. The analysis produced clear, actionable insights about timing, previous outcomes, contact channel and call frequency. Finally, the model was packaged as a validated REST API and an interactive dashboard with automated tests, turning the data mining results into a tool that marketing staff can use directly to plan more efficient campaigns.

References

- [1] Moro, S., Cortez, P. and Rita, P. (2014) 'A data-driven approach to predict the success of bank telemarketing', *Decision Support Systems*, 62, pp. 22–31. doi:10.1016/j.dss.2014.03.001.
- [2] Moro, S., Rita, P. and Cortez, P. (2014) Bank Marketing [Dataset]. UCI Machine Learning Repository. doi:10.24432/C5K306. Available at: <https://archive.ics.uci.edu/dataset/222/bank+marketing>.
- [3] Chapman, P. et al. (2000) CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.
- [4] Han, J., Kamber, M. and Pei, J. (2012) *Data Mining: Concepts and Techniques*. 3rd edn. Morgan Kaufmann.
- [5] Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- [6] Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32.
- [7] Friedman, J.H. (2001) 'Greedy function approximation: a gradient boosting machine', *Annals of Statistics*, 29(5), pp. 1189–1232.
- [8] Ke, G. et al. (2017) 'LightGBM: A highly efficient gradient boosting decision tree', *Advances in Neural Information Processing Systems*, 30.
- [9] Saito, T. and Rehmsmeier, M. (2015) 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets', *PLoS ONE*, 10(3), e0118432.
- [10] He, H. and Garcia, E.A. (2009) 'Learning from imbalanced data', *IEEE Transactions on Knowledge and Data Engineering*, 21(9), pp. 1263–1284.

[11] FastAPI (2024) FastAPI documentation. Available at: <https://fastapi.tiangolo.com>.

[12] React (2024) React documentation. Available at: <https://react.dev>.

Appendix A – Per-model Confusion Matrices

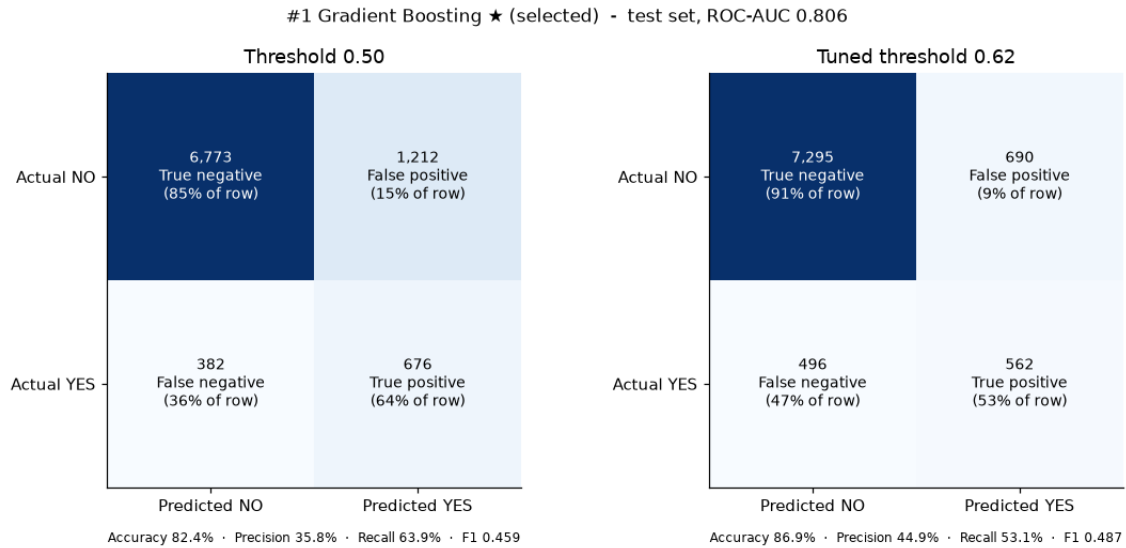


Figure 24: Confusion matrix – Gradient Boosting

#2 Random Forest - test set, ROC-AUC 0.797

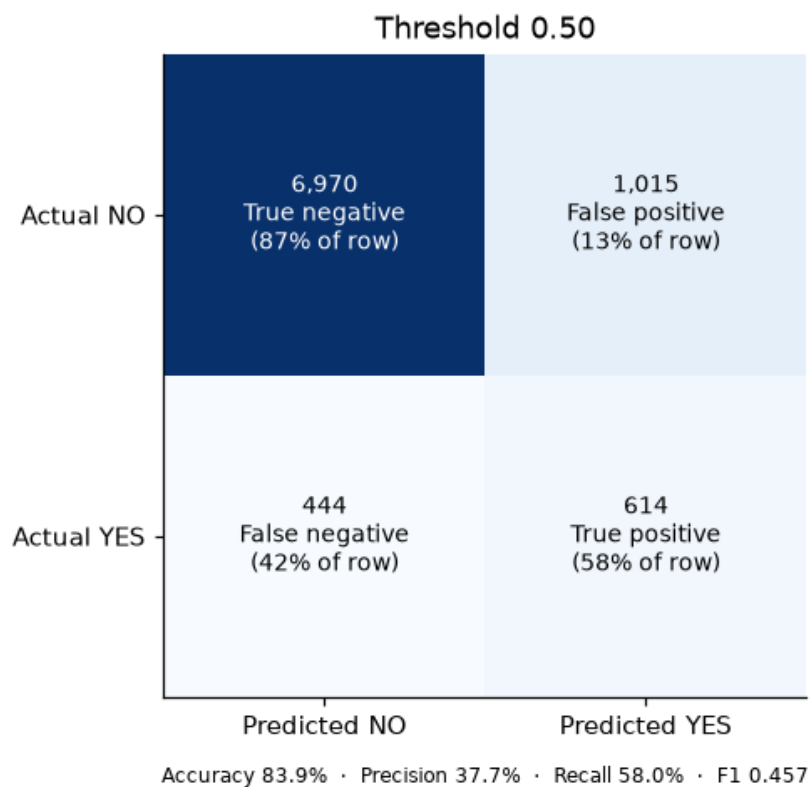


Figure 25: Confusion matrix – Random Forest

#3 Logistic Regression - test set, ROC-AUC 0.776

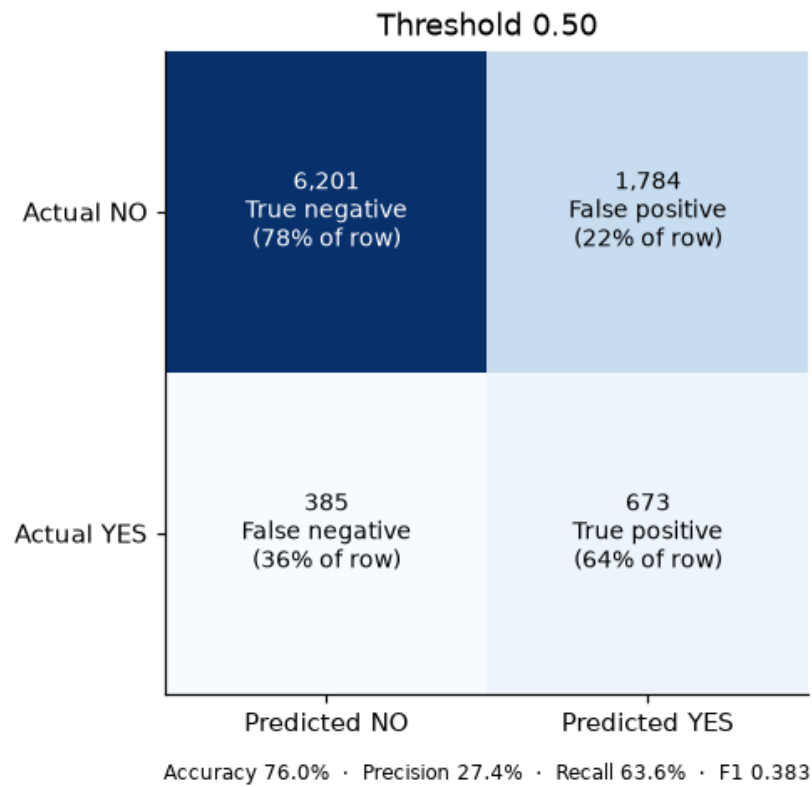


Figure 26: Confusion matrix – Logistic Regression

#4 Decision Tree - test set, ROC-AUC 0.773

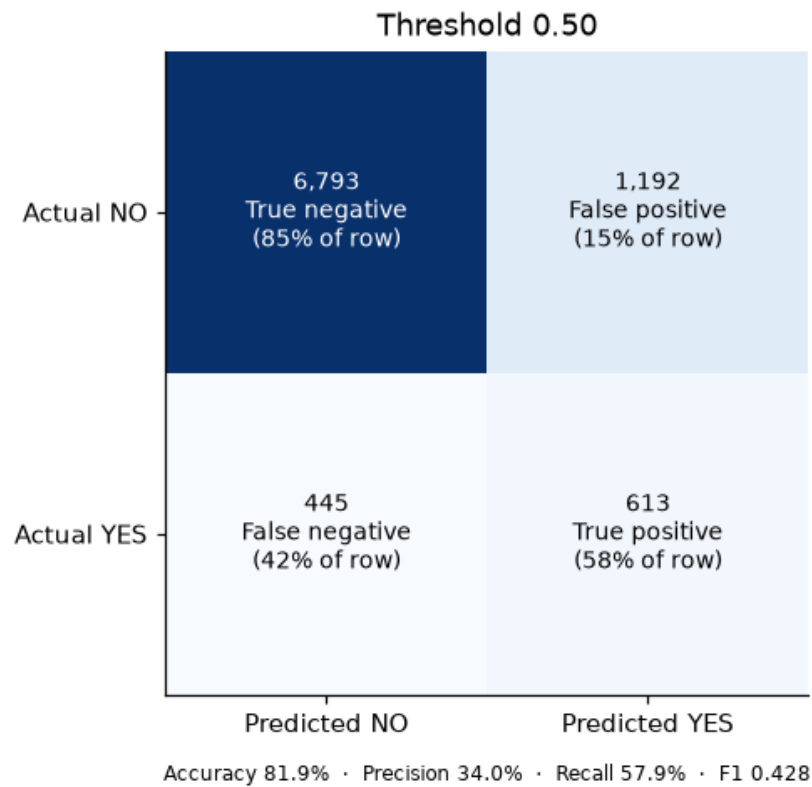


Figure 27: Confusion matrix – Decision Tree

#5 K-Nearest Neighbours - test set, ROC-AUC 0.767

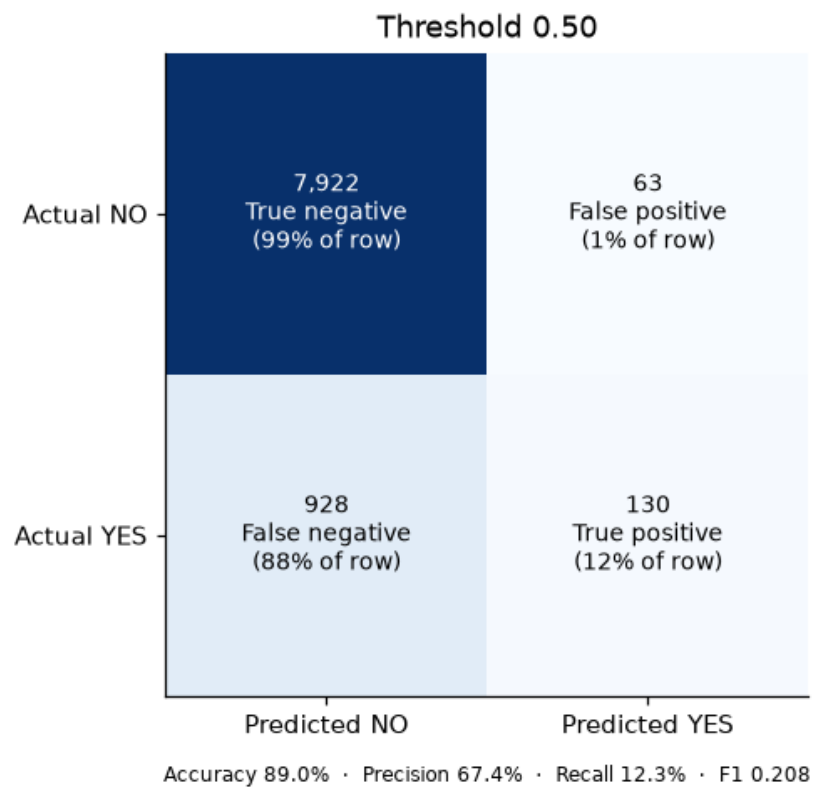


Figure 28: Confusion matrix – K Nearest Neighbours

#6 Naive Bayes - test set, ROC-AUC 0.752

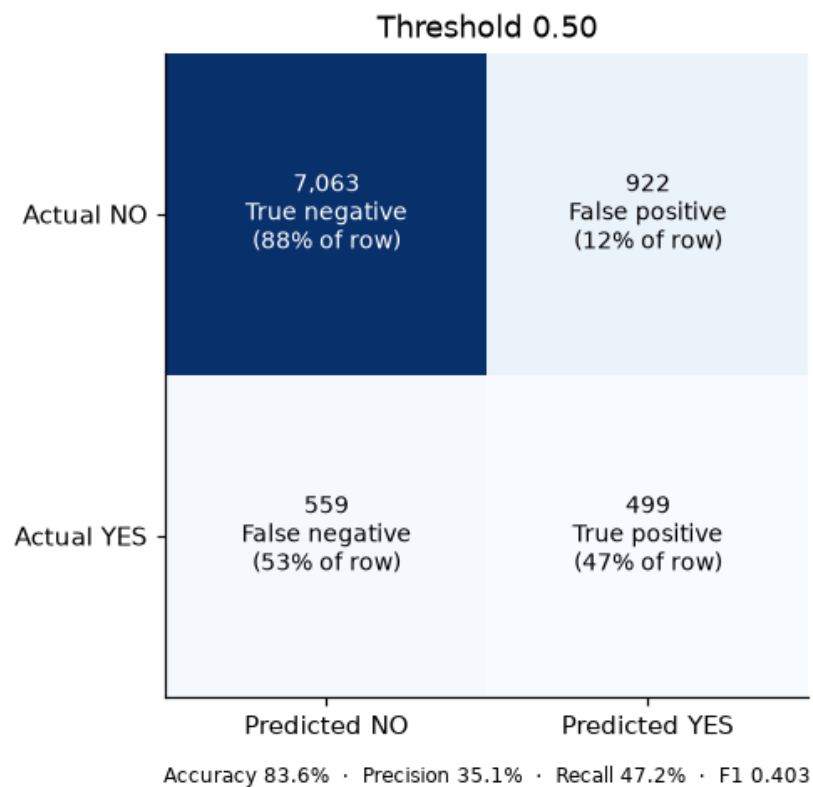


Figure 29: Confusion matrix – Naive Bayes

Appendix B – Configuration and Reproducibility

Table 18: Pipeline configuration

Setting	Value
Random seed	42
Train / test split	80 / 20, stratified
Cross-validation	3-fold stratified, shuffled
Selection metric	Cross-validated ROC-AUC
Threshold rule	Maximise F1 on out-of-fold training predictions
Outlier capping	1st / 99th percentile (training data)
Permutation importance	5 repeats, 4,000-row test sample, ROC-AUC
Selected model	Gradient Boosting (learning_rate=0.05, max_depth=8)
Decision threshold	0.62
Model trained at	2026-10-04T07:24:56+00:00